Université Abdelmalek Essaâdi Écoles National des Sciences

Appliquées – Al Hoceima

COMPTE RENDU

TP : Deep Learning

Régularisation et Hyper-paramètres

Pr. Mohamed CHERRADI
TDIA2-S4 / 2025-26

Hiba EL ouazi

1 Introduction

Objectifetcontextedu TP

Ce compte rendu présente la réalisation du TP portant sur la **Régularisation** et les **Hyper-paramètres** des réseaux de neurones, dans le cadre du module **Deep Learning**. L'objectif est d'implémenter et comparer différentes techniques de régularisation (L1, L2, Dropout) et d'analyser l'effet des hyper-paramètres d'optimisation et d'architecture sur les performances d'un modèle de classification.

Objectifs du TP

→ Comprendre et implémenter la **régularisation L1 (Lasso)** et **L2 (Ridge)**.

→ Maîtriser la technique de **Dropout** comme régulariseur stochastique.

→ Analyser l'impact du **pas d'apprentissage**, du **batch size** et du nombre d'**époques**.

→ Comparer les optimiseurs **SGD** et **Adam** sur la convergence.

→ Étudier les hyper-paramètres d'**architecture** : nombre de couches, neurones, activation.

→ Visualiser et interpréter les courbes de **perte** et d'**accuracy**.

2 Hyper-paramètres

Classification et rôles

Les hyper-paramètres se divisent en deux catégories principales : les hyper-paramètres **d'optimisation**, qui contrôlent l'algorithme d'apprentissage, et les hyper-paramètres **d'architecture**, qui définissent la structure du réseau de neurones.

Hyper-paramètres d'Optimisation

Hyper-paramètre	Valeurs typiques	Rôle
Pas d'apprentissage (lr)	0.1, 0.01, 0.001	Amplitude des mises à jour des poids
Nombre d'itérations	50 – 1000	Nombre de passages sur les données
Méthode d'optimisation	SGD, Adam, RMSProp	Algorithme de descente du gradient
Batch size	16, 32, 64, 128	Taille des mini-lots par itération
Nombre d'époques	10 – 500	Passages complets sur le dataset
λ (régularisation)	0.0001 – 0.1	Force de la pénalité L1 ou L2

Hyper-paramètres d'Architecture

Hyper-paramètre	Options disponibles	Impact
Nombre de couches	1 – 10+	Profondeur et capacité du réseau
Neurones par couche	16 – 1024	Largeur et expressivité
Type de couche	FC, CNN, RNN, LSTM, GRU	Nature des transformations apprises
Fonction d'activation	ReLU, Sigmoid, Tanh, GELU	Non-linéarité introduite
Dropout	0.0 – 0.8	Régularisation par désactivation aléatoire
Architecture spécifique	ResNet, VGG, BERT	Transfer learning et structures prédéfinies

Impact du Taux d'Apprentissage

Le **taux d'apprentissage (α)** est souvent considéré comme le plus important des hyper-paramètres. Il contrôle l'amplitude des pas de mise à jour des poids lors de la descente du gradient.

Valeur de α	Comportement	Conséquence
α trop grand (ex: 1.0)	Oscillations, divergence	La loss augmente ou oscille sans converger
α légèrement grand (ex: 0.1)	Convergence rapide mais instable	Risque de sauter au-delà du minimum optimal
α optimal (ex: 0.001 Adam)	Convergence lisse et rapide	Meilleur compromis vitesse / stabilité
α trop petit (ex: 0.00001)	Convergence très lente	Entraînement extrêmement long, résultats similaires au final

3 Implémentation

Architecture du réseau et structure du code

Architecture Choisie

Couche	Type	Taille	Activation	Justification
Entrée (Input)	Données brutes	20 neurones		20 features du dataset sklearn
Couche cachée 1	Fully-Connected	64 neurones	ReLU	Extraction de features de bas niveau
Couche cachée 2	Fully-Connected	32 neurones	ReLU	Abstraction intermédiaire + compression
Sortie (Output)	Fully-Connected	2 neurones	Softmax	Classification binaire (2 classes)

L'implémentation repose sur un réseau de neurones **fully-connected** développé entièrement en **NumPy**, sans framework externe. La classe **NeuralNetwork** intègre la propagation avant, la rétropropagation, la régularisation L1/L2, le Dropout (inverted), et les optimiseurs SGD et Adam.

Structure du Projet

Classe	Rôle
NeuralNetwork	Classe principale : Forward, Backward, Update
_init_weights()	Initialisation He (ReLU) / Xavier
_forward()	Propagation avant + Dropout inverted
_compute_loss()	Entropie croisée + terme de régularisation L1/L2
_backward()	Rétropropagation du gradient
_update_params()	SGD ou Adam optimizer
l2_regularization_demo()	Expérience avec λ variable - L2
l1_regularization_demo()	Expérience avec λ variable - L1
dropout_demo()	Expérience avec taux de Dropout variable
compare_optimizers()	Comparaison SGD vs Adam

Extrait d Implémentation du Dropout (Inverted)

```
if training and self.dropout_rate > 0:        (variable) dropout_rate: float
    mask = (np.random.rand(*A.shape) > self.dropout_rate)
    A   *= mask / (1 - self.dropout_rate)
    cache[f'mask{l}'] = mask
```

Extrait d Optimiseur Adam

```
if self.optimizer == 'adam':
    self.t += 1
    beta1, beta2, eps = 0.9, 0.999, 1e-8
    for l in range(1, L + 1):
        for p in ['W', 'b']:
            key = f'{p}{l}'
            g   = grads[f'd{key}']
            self.m[key] = beta1 * self.m[key] + (1 - beta1) * g
            self.v[key] = beta2 * self.v[key] + (1 - beta2) * g ** 2
            m_hat = self.m[key] / (1 - beta1 ** self.t)
            v_hat = self.v[key] / (1 - beta2 ** self.t)
            self.params[key] -= self.lr * m_hat / (np.sqrt(v_hat) + eps)
```

4 Régularisation

L1,L2

La **régularisation** est une technique qui ajoute un terme de pénalité à la fonction de coût pour décourager les poids trop grands, réduisant ainsi la complexité effective du modèle. L'idée est que des poids de grande amplitude signalent souvent un modèle qui sur-apprend les fluctuations du dataset d'entraînement.

Régularisation L2 (Ridge)

La régularisation L2 ajoute une pénalité proportionnelle au **carré** des poids à la fonction de coût. Elle pousse les poids vers zéro sans les annuler complètement, réduisant ainsi la variance du modèle.

Formule : $J_L2 = J + (\lambda / 2m) \times \Sigma \, ||W||^2$

Résultats expérimentaux

```
[1] Régularisation L2
  λ=0.0001 → Train Acc=1.0000 | Val Acc=0.9633
  λ=0.0010 → Train Acc=1.0000 | Val Acc=0.9633
  λ=0.0100 → Train Acc=1.0000 | Val Acc=0.9567
  λ=0.1000 → Train Acc=0.9993 | Val Acc=0.9700
```

Analyse : On observe qu'avec λ=0.1, bien que l'accuracy d'entraînement baisse légèrement (99.93% vs 100%), l'accuracy de validation est la meilleure (97.00%). Cela illustre parfaitement le principe de régularisation : accepter un léger sacrifice sur les données d'entraînement pour gagner en généralisation.

Régularisation L1 (Lasso)

La régularisation L1 pénalise la **valeur absolue** des poids, favorisant la **parcimonie** (sparsity) : certains poids sont exactement annulés, ce qui réalise une sélection implicite de caractéristiques.

Formule : $J_L1 = J + (\lambda / m) \times \Sigma \, ||W||$

Résultats expérimentaux

```
[2] Régularisation L1
  λ=0.0001 → Train Acc=1.0000 | Val Acc=0.9633
  λ=0.0010 → Train Acc=1.0000 | Val Acc=0.9633
  λ=0.0100 → Train Acc=1.0000 | Val Acc=0.9600
  λ=0.1000 → Train Acc=0.9821 | Val Acc=0.9700
```

Contrairement à L2, le gradient L1 a une amplitude constante (λ/m), ce qui fait 'tirer' les poids vers zéro avec la même force, quelle que soit leur amplitude. Pour les petits poids, cette force est suffisante pour les annuler complètement → sparsité.

Comparaison L1 vs L2

Critère	L1 (Lasso)	L2 (Ridge)
Pénalité	Valeur absolue : $\Sigma\|W\|$	Carré : ΣW^2
Gradient	$(\lambda/m) \times \text{sign}(W)$	$(\lambda/m) \times W$
Effet sur les poids	Annulation exacte de certains poids	Poids petits mais rarement nuls
Parcimonie (Sparsity)	Oui –:sélection de features	Non :tous les poids restent actifs
Robustesse aux outliers	Plus robuste	Moins robuste
Usage recommandé	Features nombreuses, sélection utile	Usage général, modèles denses
Complexité du gradient	Non-différentiable en 0 (sign)	Toujours différentiable

7 Dropout

Régularisation stochastique

Le Dropout est une technique de régularisation révolutionnaire proposée par Srivastava et al. (2014). À chaque itération d'entraînement, chaque neurone des couches cachées est désactivé (mis à zéro) avec une probabilité p, indépendamment des autres. Cette désactivation aléatoire force le réseau à développer des représentations redondantes et robustes, car il ne peut jamais s'appuyer systématiquement sur un sous-ensemble particulier de neurones.

Principe du Dropout Inverted

Nous utilisons la variante **Inverted Dropout**, qui est la plus courante en pratique. Son avantage clé : **aucune modification nécessaire lors de l'inférence**. Le rescaling est effectué durant l'entraînement, pas au test.

Phase d'entraînement : mask ~ Bernoulli(1-p) ; A_drop = A × mask / (1-p)

Phase d'inférence : A_test = A (aucune modification nécessaire)

```
if training and self.dropout_rate > 0:        (variable) dropout_rate: float
    mask = (np.random.rand(*A.shape) > self.dropout_rate)
    A   *= mask / (1 - self.dropout_rate)
    cache[f'mask{l}'] = mask
```

Interprétation : Ensemble de Sous-Réseaux

Une façon puissante d'interpréter le Dropout est de le voir comme l'**entraînement simultané d'un ensemble exponentiel de sous-réseaux**. Avec N neurones et un taux p, il existe 2^N configurations possibles de masques. En pratique, le réseau final est une sorte de **moyenne bayésienne** de tous ces sous-réseaux, ce qui explique ses meilleures performances en généralisation.

Résultats Expérimentaux

```
[3] Dropout
  Dropout=0.0 → Train Acc=1.0000 | Val Acc=0.9733
  Dropout=0.2 → Train Acc=1.0000 | Val Acc=0.9900
  Dropout=0.4 → Train Acc=0.9957 | Val Acc=0.9900
  Dropout=0.6 → Train Acc=0.9886 | Val Acc=0.9800
```

5 Résultats et Visualisations

Analyse comparative

Graphiques Comparatifs

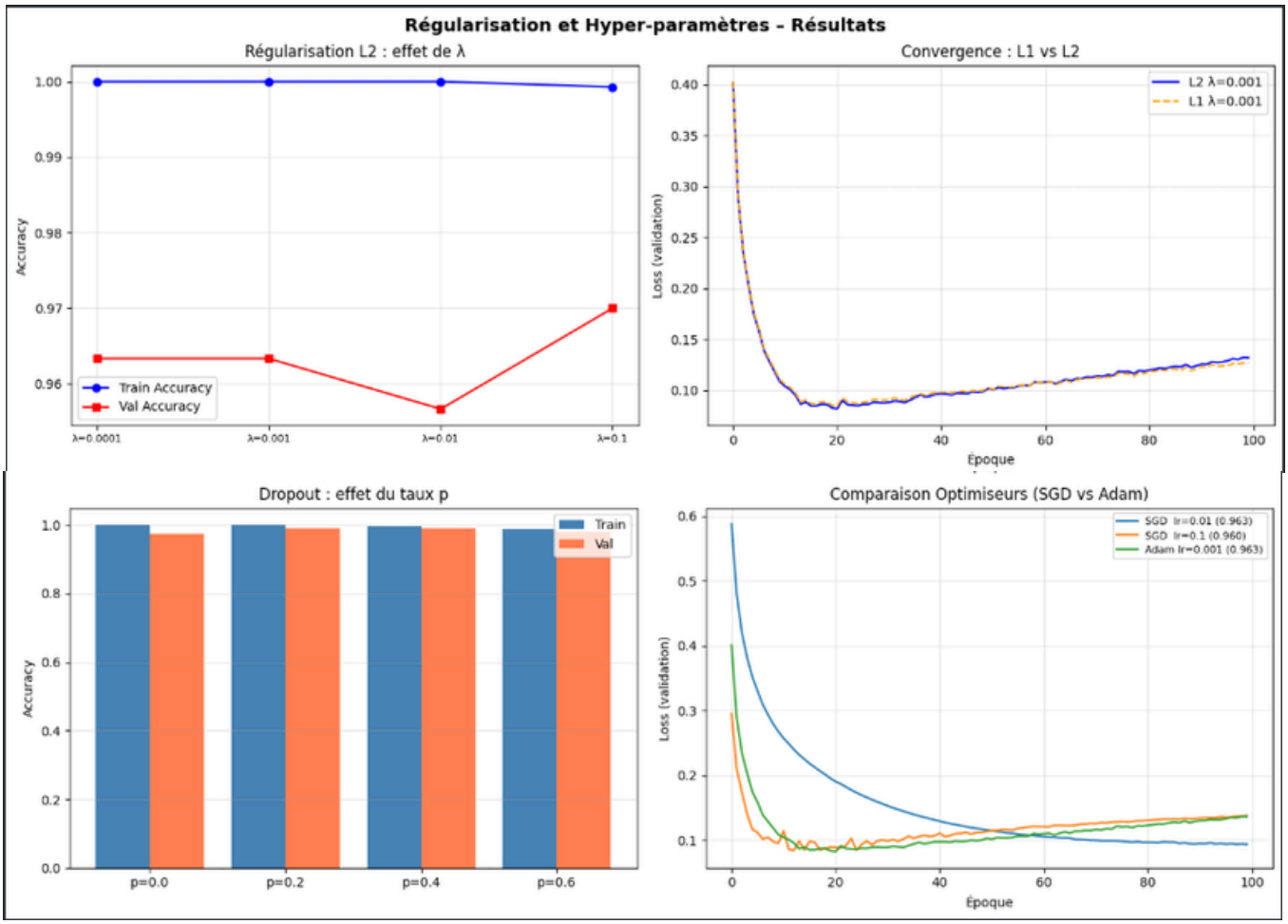

Figure : Comparaison L2, L1 convergence, Dropout et optimiseurs

Comparaison des Optimiseurs (SGD vs Adam)

Le choix de l'**algorithme d'optimisation** a un impact majeur sur la vitesse de convergence et la qualité de la solution finale. Nous comparons ici les deux optimiseurs implémentés : **SGD classique** et **Adam** (Adaptive Moment Estimation).

SGD :Stochastic Gradient Descent

Le SGD est la forme la plus simple d'optimisation par gradient. À chaque itération, les paramètres sont mis à jour dans la direction opposée au gradient, avec un pas fixe α. Le SGD avec mini-batch est le standard de base auquel tous les autres optimiseurs sont comparés.

Avantages : Simple, peu de mémoire, bonne généralisation parfois.

Inconvénients : Sensible au taux d'apprentissage, convergence lente dans les ravins de la loss surface, oscillations.

Adam :Adaptive Moment Estimation

Adam (Kingma & Ba, 2014) combine les avantages de RMSProp et Momentum. Il maintient une **moyenne mobile des gradients (1er moment)** et une **moyenne mobile des carrés des gradients (2ème moment)**, et applique une **correction de biais** pour les premières itérations.

Résultats Comparatifs

Optimiseur	Learning Rate	Val Accuracy	Vitesse de convergence
SGD	0.01	96.33 %	Lente, stable
SGD	0.1	96.00 %	Rapide mais instable
Adam	0.001	96.33 %	Rapide et stable

Interprétation des 4 Graphiques

Graph1: Régularisation L2 (Effet de λ) : La courbe de Val Accuracy montre un **optimum en λ=0.1**, confirmant que la régularisation améliore la généralisation. L'accuracy d'entraînement reste proche de 100% pour toutes les valeurs testées, ce qui indique que le modèle a suffisamment de capacité pour fitter les données.

Graph2 : Convergence L1 vs L2 : Les deux régulariseurs convergent vers des niveaux de loss similaires, mais L2 montre une courbe légèrement plus lisse. L1, avec son gradient de norme constante (signe), peut induire de légères oscillations autour du minimum.

Graph3 :Dropout (Effet du taux p) : Le graphique en barres révèle clairement que p=0.2 et p=0.4 offrent les meilleures Val Accuracy (99%), tout en réduisant l'écart avec Train Accuracy. P=0.6 commence à dégrader les performances.

Graph4 :Optimiseurs (SGD vs Adam) : Adam converge beaucoup plus rapidement que SGD. Dès les 20 premières époques, Adam atteint une loss proche de sa valeur finale, tandis que SGD continue de décroître lentement. La courbe d'Adam est également plus régulière.

6 Conclusion

Synthèse des résultats

Ce TP a permis de mettre en pratique le cycle complet de la régularisation dans les réseaux de neurones. L'implémentation from scratch a renforcé la compréhension des mécanismes internes. Le **Dropout avec p=0.2–0.4** s'est révélé le régulariseur le plus efficace sur ce jeu de données, surpassant L1 et L2 en termes de Val Accuracy. L'optimiseur **Adam** offre la meilleure stabilité de convergence.